

Mohammad Hamdy Ghanem

SMALL VISUAL BASIC

3.1

**Priručnik za jezik
2025.**

Sa engleskog preveo: dr Aca Cvetković, jun, 2025.

1. Zdravo, Small Visual Basic!

Šta je Small Visual Basic?

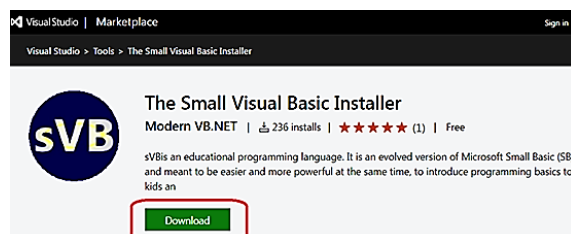
Small Visual Basic (sVB) je obrazovni programski jezik, koji je kreirao inženjer Mohamed Hamdi kao evoluiranu verziju Microsoft Small Basic-a (SB). Namenjen je da bude lakši i istovremeno moćniji, kako bi upoznao osnove programiranja decu i početnike bilo kog uzrasta, pod uslovom da mogu da koriste englesku tastaturu na Windows operativnom sistemu.

Da bi se postigao ovaj cilj, sVB ima mnogo novih delova koji ne postoje u Small Basic-u, kao što su dizajner obrazaca, biblioteka malih Windows obrazaca, debager i mnoge nove sintaksne funkcije i poboljšanja kompajlera i uređivača koda. Za više detalja pogledajte *Zašto nam je potreban sVB?* na stranicama niže.

Ukratko, može se reći da je sVB Visual verzija Small Basic-a i mala verzija Visual Basic-a.

Preuzmite program

Počevši od sVB 2.8.1, jezik ima instalater koji možete preuzeti sa <https://marketplace.visualstudio.com/items?itemName=ModernVBNET.sVBInstaller>, samo kliknite na dugme za preuzimanje Download, zatim dvaput kliknite na preuzetu zip datoteku, raspakujte je i dvaput kliknite na *svb installer.msi* datoteku.

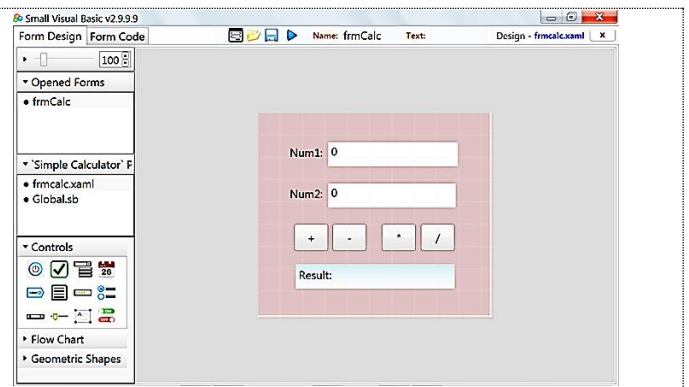
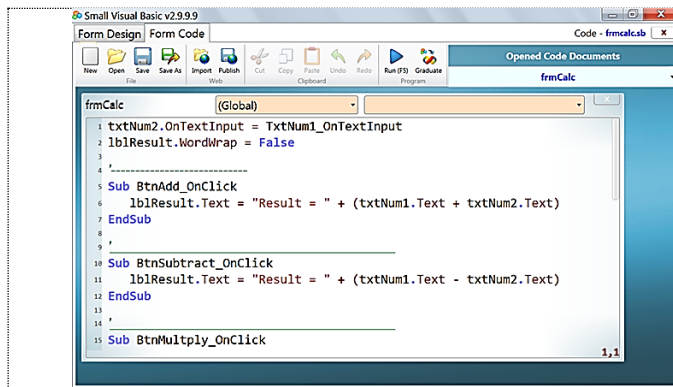


Kada se pokrene instalacioni program, uveriće se da vaš računar ima .NET Framework 4.8, a ako ne, zatražiće od vas da ga preuzmete i instalirate preko datog linka.

Nakon toga možete pritisnuti sledeće na svakoj stranici za podešavanje da biste instalirali sVB sa podrazumevanim opcijama. Sačekajte da se podešavanje završi za nekoliko sekundi i zatvorite instalater kada poslednja stranica potvrdi da je podešavanje uspešno završeno. Sada imate dve ikone za sVB i njegovu PDF knjigu na radnoj površini i u fascikli Small Visual Basic u meniju „Start“, tako da možete kliknuti na njih da biste pokrenuli sVB i otvorili njegov kompletan priručnik (ovu knjigu!). Čestitamo, sada možete početi sa programiranjem.

Isprobajte primere

Nakon instaliranja Small Visual Basic-a, možete dvaput kliknuti na ikonu „sVB Samples“ – *sVB primeri* na radnoj površini da biste otvorili fasciklu sVB primeri, koja je instalirana u vašoj fascikli sa dokumentima sa oko 90 primera, uključujući igre Lopta, Tom i Džeri, Automobili i Tetris, i mnoge druge zanimljive primere.



Da biste isprobali bilo koji primer, otvorite njegovu fasciklu i samo dvaput kliknite na bilo koju .sb datoteku (onu koja ima sVB ikonu) da biste je otvorili u sVB IDE-u, što će prikazati kôd ove datoteke u „Form Code“ – uređivaču koda.

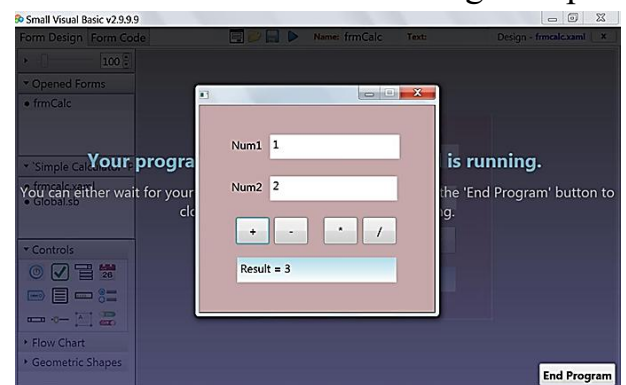
Ako je datoteka povezana sa formularom – formom, ona će takođe biti učitana u dizajner formulara, a možete preći na nju klikom na karticu „Form Design“ – *Dizajn formulara* na vrhu IDE-a.

Prethodne 2 slike vam pokazuju kako će uređivač koda i dizajner formulara izgledati kada otvorite aplikaciju Simple Calculator – Jednostavan kalkulator iz fascikle primeri.

Takođe možete otvoriti bilo koji kôd ili datoteku formulara iz IDE-a klikom na dugme Open – *Otvori* u gornjoj traci sa alatima ili pritiskom na Ctrl+F5 na tastaturi.

Sada, da biste pokrenuli projekat, kliknite na dugme „Run“ – *Pokreni* u traci sa alatima (ili pritisnite F5 na tastaturi). Ovo će prikazati formular na ekranu, gde možete interagovati sa njegovim kontrolama da biste unosili podatke i dobijali rezultate onako kako je formular programiran da radi.

Imajte na umu da će zatvaranje glavnog obrasca ili klik na dugme „End Program“ – *Završi program* zatvoriti program i vratiti vas u IDE.



Korišćenje sVB izvornog koda

sVB je projekat otvorenog koda, objavljen kao GitHub repozitorijum.

Izvorni kod MS Small Basic-a je napisan u C#, ali sam konvertovao kopiju u VB.NET i koristim ga za kreiranje sVB-a.

Ja sam profesionalni C# programer, ali mi se činilo neprikladnim da pišem BASIC kompajler sa C jezikom (a da ne spominjem da je VB.NET moj omiljeni jezik).

Štaviše, želeo sam da koristim sVB kao poligon za razumevanje i rad sa kompajlerima, kako za mene, tako i za VB.NET zajednicu. Ovaj mali kompajler sa malim alatima pruža veoma jednostavan način da se razume kako se kompajleri grade i kako rade, što može olakšati rad sa VB.NET kompajlerom u sledećem koraku (koji je deo Roslyn-a).

Ovo je neophodan korak za VB zajednicu, nakon što je MS zanemario VB.NET od 2017. godine i prestao da ga razvija od 2020. godine.

I zato je Entoni D. Grin napravio odvojen Roslyn i počeo da razvija VB.NET pod imenom ModVB od 2022. godine.

Entoni je učestvovao u dizajniranju i kreiranju Roslyn-a i VB.NET-a, ali on je i dalje jedan

čovjek i zajednica mora da mu pomogne da održava i razvija ModVB, ako zaista žele da dugo živi i napreduje.

Dakle, evo malog programskog jezika napisanog u VB.NET-u za vas. Možete ga forknuti (razvijati u drugu stranu, razvijati dalje pod rugim imenom), igrati se sa njim i razvijati ga kako želite, a kada se osećate prijatno sa kompajlerima, pokušajte da pogledate Roslyn i ModVB.

Imajte na umu da su svi sVB projekti WPF projekti, koji ciljaju .NET framework 4.5. Možete pokrenuti izvorni kôd u VS.NET 2019 i kasnijim verzijama. Ali pre pokretanja koda, kopirajte fascikle „Lib“ i „Toolbar“ iz fascikle „SmallBasicIDE\SB.Lib“ u fascikle „SmallBasicIDE\bin\Debug“ i „SmallBasicIDE\bin\Release“, jer očigledno „Git“ isključuje ove fascikle, a ja to više volim na ovaj način.

Zašto nam je potreban sVB

„Bejsik je nekada bio na svakom računaru koga se dete dotaklo, ali danas nema lakog načina da se deca zainteresuju za programiranje.“

Dejvid Brin, „Zašto Džoni ne može da programira“, 2006.

Jednog dana, Vidžaje Radži je pročitao gornji članak, pa je odlučio da nešto uradi kako bi rešio taj problem. U slobodno vreme u Majkrosoftu, kreirao je Smart Bejsik kao obrazovni programski jezik za decu od 7 godina i više, a Majkrosoft ga je objavio 2008. godine.

Nisam bio pročitao ovaj članak, niti sam znao da Smart Bejsik postoji, kada sam pokušao da naučim VB.NET svog 13-godišnjeg nećaka, što nije bilo lako za njega!

Uzgred, i on se zove Mohamed Hamdi Ganem (mlađi), jer je moj brat dobio ime po našem ocu, a njegov sin je dobio ime po meni!

Zato sam se našao da i ja postavljam slično pitanje: Zašto Mohamed ne može da programira?

Postoji mnoštvo fundamentalnih činjenica o VB.NET-u koje moraju da se nauče pre nego što se napravi bilo šta korisno. Osim toga, stalno me je pitao ko bi koristio tako jednostavne primere u stvarnom svetu, i bio je potpuno u pravu! Dakle, morao sam da napravim jednostavnu igru trkačkih automobila da bih privukao njegovu pažnju, ali stvari nisu bile tako jednostavne za njega! Korišničke kontrole, objekti i nasleđivanje?! Apsolutno se preopteretio!

Sećam se da je klasični Visual Basic bio mnogo lakši za početnika, čak i za nekoga bez ikakvog programerskog obrazovanja, ali VB.NET je postajao komplikovan tokom godina kako je postajao moćniji. Kada sam o tome razgovarao sa VB zajednicom, jedan programer je predložio da umesto toga koristim Small Basic, i to je bio prvi put da čujem za to. Pokušao sam to da uradim sledeće godine sa svojim mlađim nećacima, Alijem i Omarom, i oni su se već zainteresovali za Small Basic. Small Basic je zaista mali, sadrži samo 14 ključnih reči za izvršavanje osnovnih programskih instrukcija kao što su naredbe „Sub“, „If“, „For“, „While“ i „Goto“. Small Basic je dinamičan jezik, jer ne deklariše promenljive određenih tipova. Samo dodelite vrednost validnom identifikatoru, a VB.NET ga deklariše kao primitivnu promenljivu, koja može da sadrži string, broj ili niz. Ovo čini jezik veoma lakim za učenje i korišćenje za decu. Dakle, Ali i Omar su bili zadovoljni sa SB, sve dok nisu stigli do crteža grafičkih prozora koji zavise od sinusnih i kosinusnih funkcija, što ih je zbunilo, jer su još uvek bili u 4. razredu bez ikakvog trigonometrijskog znanja! PDF knjiga koja dolazi uz SB otežava crtanje fokusirajući se na crtanje oblika pomoću trigonometrijskih funkcija (čak sadrži i primer fraktala!).

Ovo nije najbolji način da se deci predstavi programiranje. Crni komandni prozor (tekst

prozor) je jednostavan, ali dosadan, dok je korišćenje vektorske grafike ili crtanje pomoću kornjače na grafičkom prozoru neverovatno, ali može biti prilično teško. Dobra vest je da vam klasa Controls omogućava da nacrtate TextBox i Button na grafičkom prozoru, da upravljate njihovim svojstvima i da upravljate njihovim događajima. Ali, nažalost, dete mora da dizajnira formu na slepo dok dodaje kontrole pomoću koda. Štaviše, kôd koji se koristi za komunikaciju sa ovim kontrolama je opširan, jer SB nema objekte, tako da možete samo da sačuvate ime kontrole u promenljivoj, a zatim je pošaljete statičkim/deljenim metodama da biste promenili njena svojstva ili izvršili zadatke. Na primer:

```
Btn = Controls.AddButton("Enable", 100, 100)
Controls.ButtonClicked = OnClick
Sub OnClick
If Controls.GetButtonCaption(Btn) = "Enable" Then
Controls.SetButtonCaption(Btn, "Disable")
Else
Controls.SetButtonCaption(Btn, "Enable")
EndIf
EndSub
```

Ovo nije vrsta koda koju želite da pokažete detetu!

U stvari, biće mu lakše naučiti Visual Basic, tako da može da prevuče dugme iz alatke, ispusti ga na prozor, postavi mu ime i natpis iz prozora sa svojstvima, dvaput klikne na njega da bi otišao na njegovu funkciju za obradu događaja klika u uređivaču koda i jednostavno napiše:

```
If btn.Text = "Enable"
btn.Text = "Disable"
Else
btn.Text = "Enable"
EndIf
```

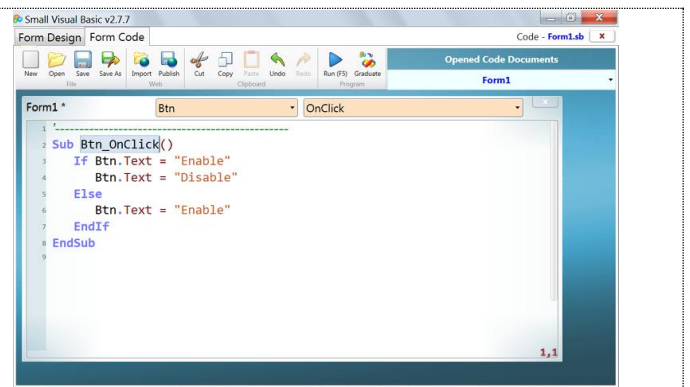
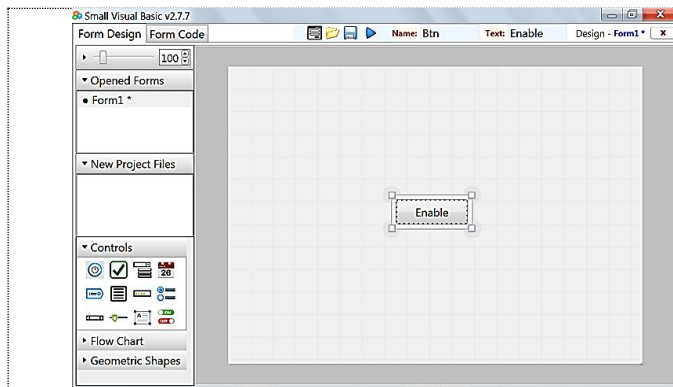
I to je to. Brz, čist, lak i kratak kôd, koji nas je naterao da zavolimo programiranje! Neverovatno je da je SB zakomplikovao tako lak zadatak, u ime jednostavnosti i lakih za učenje za decu! Probao sam neka alternativna SB IDE-a, ali su ili:

- **složenija** (previše napredna da bi se ništa važno uradilo sa jezikom koji je namenjen da bude igračka za učenje),

- ili dovoljno **jednostavna** da se nacrtaju kontrole i generiše neki kôd za njih, ali i dalje ne mogu da prevaziđu ograničenja SB sintakse pri radu sa objektima.

Zato sam se pitao: Zašto SB ima samo dva prozora? Zar SB ne može da ima dizajner obrazaca i jednostavniju sintaksu za programiranje kontrola? Zapravo sam ovo tražio u SB repozitorijumu na GitHub-u, ali nisam dobio pažnju tima, pa sam odlučio da sam dokažem koncept. Dodao sam dizajner obrazaca u SB, koji sam napravio koristeći alatku pod nazivom „Diagram Helper“ koju sam kreirao još 2014. godine za dizajniranje dijagrama toka, ali sam otkrio da se može lako modifikovati da radi kao dizajner obrazaca. Takođe sam dodao malu WinForms biblioteku u SB i napisao prekompiletor kako bih spustio sintaksu objekata na normalnu sintaksu koju SB kompajler razume. Koristeći ovaj jednostavan trik, omogućio sam korišćenje imena kontrola kao objekata kako bi kôd bio kraći i lakši.

Tako sam završio sa Visual Small Basic-om koji je bio toliko blizak Visual Basic-u da se može smatrati njegovom smanjenom verzijom, pa sam odlučio da ga nazovem Small Visual Basic.



Objavio sam preliminarnu verziju sVB-a na GitHub-u u januaru 2021. i pozvao SB tim da iskoristi ideju za evoluciju SB-a, ali nisu delovali zainteresovano!

Oko 5 meseci nisam imao nameru da radim bilo šta dalje, ali u junu 2021. godine, moja voljena baka Om Vahba je umrla, neka joj Bog dušu prosti, pa sam odlučio da završim sVB u čast njenog sećanja.

Takođe u to vreme, moj nećak Mohamed Mohsen je počeo da gleda moje YouTube video snimke o SB-u, i želeo sam da ga poštedim problema sa SB-om koje sam gore objasnio, pa sam vredno radio da objavim prvu stabilnu verziju sVB-a u julu 2021. godine, koja je imala samo 3 kontrole u alatima!

Nastavio sam da ažuriram jezik tokom naredne dve godine, sve dok nije stigao do v3.1 sa objavljivanjem ove knjige.

I to je bila priča o četvorogodišnjem putovanju od Small Basic-a do Small Visual Basic-a.

To je takođe Small Visual Basic .NET!

Može se reći:

Small Visual Basic = Small Basic + Visual Basic: ali to nije cela istina!

sVB je zamišljen kao srednja faza između SB i VB.NET, tako da njegova sintaksa kombinuje najbolje od SB, VB i VB.NET jezika!

Takođe je jednostavan kao Windows Forme, ali ima i neke napredne WPF grafičke funkcije!

Hajde da se malo dublje pozabavimo time:

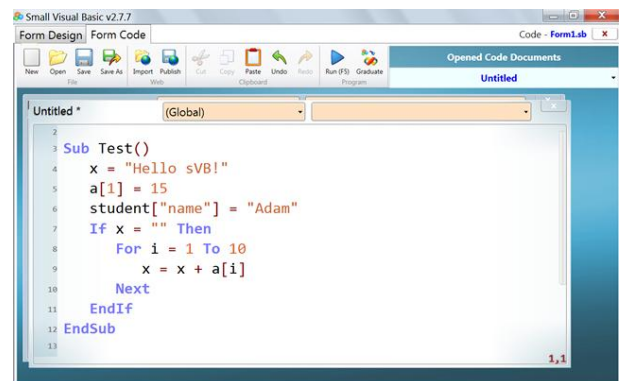
sVB i Small Basic

sVB je izgrađen na Small Basic IDE-u, kompajleru i biblioteci.

U većini slučajeva, SB kôd radi u sVB-u, ali postoji nekoliko ključnih promena koje ćemo istražiti u sledećim pasusima. Pogledajte takođe: *Kako rešiti probleme sa SB promenljivim u sVB-u* na nekim od nižih stranica.

sVB takođe uvodi neka poboljšanja u SB, kao što su:

1. SB koristi ključne reči **EndFor** i **EndWhile**, i one su i dalje validne u sVB-u, ali vam dozvoljava (i preporučuje) da umesto njih koristite ključne reči **Next** i **Wend**, kako biste bili konzistentniji sa VB-om.



2. sVB je dodao mnoge metode SB tipovima kao što su Array, Text i File, i napravio izmene u nekoliko starih metoda, poput toga da metode datoteke vraćaju True ili False umesto „SUCCESS“ ili „FAILED“. Više o tome možete saznati u delu sVB biblioteke ove knjige.

3. SB dozvoljava korišćenje razmaka kao separatora između argumenata, ali sVB to ne dozvoljava i dozvoljava samo korišćenje zareza.

4. Postoji greška u parsiranju (rasčlanjivanju) u SB koja vam dozvoljava da ne dodate završni citat literala stringa, ali je ispravljena u sVB-u.

5. Postoji greška u parsiranju u SB koja vas sprečava da koristite dvostruke navodnike za ugrađivanje citata unutar literala stringa, ali je ispravljena u sVB-u.

6. sVB ne može da koristi eksterne biblioteke kreirane za SB, osim ako nisu prepisane za sVB da bi ciljale SmallVisualBasicLibrary.dll.

Dakle, SB programi koji koriste takve biblioteke neće raditi u sVB. Dobra vest je da je najveća i najpoznatija SB eksterna biblioteka, LitDev, kreirala verziju validnu za sVB i sada je instalirana sa sVB, tako da je možete direktno koristiti ako želite.

7. sVB dodaje mnoga poboljšanja u tekstualni prozor:

a. sVB je dodao metod `TextWindow.WriteFormatted` kako bi vam omogućio lako formatiranje izlaznog teksta zamenom čuvara mesta sa stavkama datog niza.

b. sVB je dodao metod `TextWindow.WriteLine` kako bi vam omogućio direktno ispisivanje niza stavki, svaka u posebnom redu, što može biti veoma korisno kada se koristi sa sVB inicijalizatorom niza:

```
TextWindow.WriteLine(  
{"Line1", "Line2", "Line3"}  
)
```

c. Možete koristiti `TW` kao alternativno ime za `TextWindow`.

d. Takođe možete koristiti i sledeće prečice za TW metode, a uređivač koda će ih konvertovati u puno ime sa ispravnom sintaksom čim napustite liniju:

Prečica	Metoda	Primer
?	TW.WriteLine	? "Hello"
?	TW.WriteLine	? x, y
?	TW.Read	x = ?
?:	TW.Write	?: "Enter x: "
#?	TW.ReadNumber	x = #?

Za više detalja, pogledajte primere date za ove metode u *TextWindow* na stranicama niže.

8. sVB dodaje mnoga poboljšanja Grafičkom prozoru. Na primer:

a. Možete koristiti `GW` kao alternativno kraće ime za `GraphicsWindow`.

b. Možete koristiti `MsgBox` kao prečicu za prikazivanje prozora sa porukom sa podrazumevanim naslovom „Message“ – *Poruka*. Npr:

```
MsgBox „Zdravo!“
```

Kada napustite red ili sačuvate datoteku, gornji kod će biti konvertovan u:

```
Forms.ShowMessage("Zdravo!", "Poruka")    što je slično:  
GW.ShowMessage("Zdravo!", "Poruka")
```

ali `Forms.ShowMessage` ne nameće prikazivanje `GraphicsWindow`, što ga čini pogodnim za upotrebu u bilo kom obliku u projektu.

Takođe možete dati naslov za sintaksu prečice `MsgBox` ovako:

```
MsgBox „Kako ste?“, „Zdravo“
```


Što će biti konvertovano u:

```
Forms.ShowMessage("Kako ste?", „Zdravo")
```

c. U SB možete dodati samo dugmad i tekstualna polja u grafički prozor, ali u sVB možete dodati sve kontrole koje vidite u alatka dizajnera obrazaca, koristeći metode [Controls.AddX](#) kao što su [Controls.AddCheckBox](#), [Controls.AddComboBox](#), [Controls.AddDatePicker](#), itd.

Ove metode vraćaju ključeve dodatih kontrola, gde sVB može da ih tretira kao objekte ovih tipova kontrola, tako da je lako pristupiti njihovim svojstvima i metodama i dodati rukovaoce za njihove različite događaje (pogledajte primere date uz svaku metodu).

d. U SB postoji samo jedan objekat Timer, što komplikuje stvari ako treba da preduzmete različite radnje u različitim intervalima. U sVB ovo nije problem, jer možete dodati onoliko tajmera koliko vam je potrebno u grafički prozor koristeći metodu [Controls.AddTimer](#). U stvari, uređivač koda (Form Code) ovo olakšava tako što pruža komandu „Add New Timer“ – *Dodaj novi tajmer* u desnoj padajućoj listi u naslovnoj traci dokumenta koda. Prvi put kada kliknete na ovu komandu, generisaće se kôd za podešavanje inicijalizacije intervala tajmera SB i obradu njegovog događaja [Tick](#). Nakon toga, svaki put kada kliknete na tu komandu, generisaće se kôd za dodavanje novog tajmera u GW.

e. GW je dobio neke nove metode crtanja kao što su metode [DrawArc](#), [DrawBezierCurve](#), [DrawQuadraticBezierCurve](#), [DrawPolygon](#) i [FillPolygon](#). Biblioteka [Shapes](#) je takođe dobila metod [AddPolygon](#).

f. Možete kreirati kompozitni oblik koristeći geometrijsku putanju i nacrtati ga na grafičkom prozoru pozivanjem metode [Shapes.AddGeometricPath](#).

g. Možete popuniti pozadinu grafičkog prozora gradijentnom četkicom pozivanjem metode [FillGradientBackground](#). Takođe možete koristiti svojstvo [GradientEndColor](#) da biste kreirali gradijentnu četkicu koja će se koristiti za popunjavanje oblika nacrtanih na grafičkom prozoru pomoću metoda [GW.DrawX](#), [Shapes.AddX](#) ili metode [Turtle](#).

h. Možete prikazati GW u režimu celog ekrana podešavanjem svojstva [FullScreen](#) na True.

i. Možete prikazati grafiku direktno na radnoj površini podešavanjem [BackgroundColor](#) na [Colors.None](#) ili [Colors.Transparent](#).

j. Možete zadržati grafički prozor iznad svih ostalih prozora podešavanjem svojstva [Topmost](#) na True.

k. Pošto je grafički prozor zapravo normalan oblik kao i svaki drugi oblik koji dodate svom sVB projektu, možete koristiti metod [GraphicsWindow.AsForm](#) da biste dobili objekat oblika koji ga predstavlja, tako da možete pristupiti većem broju njegovih svojstava i metoda.

l. Kornjača može da popuni oblast pozivanjem metoda [CreateFigure](#) i [FillFigure](#). Takođe možete promeniti veličinu kornjače promenom njenih svojstava [Width](#) ili [Height](#), povećati njenu brzinu do 50 i izabrati da onemogućite animaciju da biste dobili trenutno kretanje podešavanjem svojstva [UseAnimation](#) na False.

m. Neki SB tipovi su dobili malo ljubavi od sVB kompajlera, poput tipova [Sound](#) i [Shape](#). Na primer, kada dodelite vrednost koju vraća metoda [Sound.Load](#) promenljivoj, možete je koristiti kao objekat tipa [Sound](#), tako da možete direktno pristupiti metodama zvuka iz nje. Za primer, pogledajte metodu [Sound.Pause](#).

Slično tome, povratna vrednost iz metoda [Shapes.AddX](#) može se tretirati kao objekti tipa [Shape](#). Za primer, pogledajte metodu [Shapes.AddLine](#).

n. Možete koristiti sVB biblioteku [Thread](#) za pokretanje potprograma u njihovim sopstvenim

nitima. Ovo može biti korisno kada crtate složenu grafiku na GW-u, posebno kada crtate pomoću metode `SetPixel`.

9. sVB uređivač koda (Form Code) olakšava sastavljanje višelinijuskog string literala. Sve što vam je potrebno je da pritisnete Enter unutar string literala, a uređivač koda će ga podeliti u dva reda i dodati potreban kôd za ugrađivanje novog linijskog stringa. Na primer, otkucate:

```
?"Linija 1
```

zatim pritisnite Enter i otkucate:

```
Linija 2
```

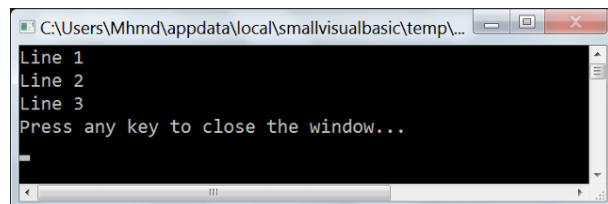
zatim pritisnite Enter i otkucate:

```
Linija 3
```

Ovo je ono što dobijate u uređivaču koda:

```
TW.WriteLine("Line 1" & Text.NewLine &  
"Line 2" & Text.NewLine &  
"Line 3")
```

A ovo je ono što dobijete kada pokrenete kôd:



sVB i Visual Basic

sVB ima mnoga poboljšanja u odnosu na SB kako bi pisanje aplikacija bilo brzo i jednostavno uz malo koda. Vraća radost i uzbuđenje korišćenja vb6 za pisanje RAD aplikacija.

Da bi to uradio, sVB sadrži malu **WinForms** biblioteku i dizajner obrazaca – Form Design koji vam omogućava da dodate mnogo obrazaca u projekat i samo jedan globalni modul. Možete prevući kontrole iz alata, ispustiti ih na obrazac i koristiti prozor sa svojstvima da biste promenili neka od njihovih svojstava. Možete koristiti dizajner menija da biste dodali meni za obradu u obrazac i dizajnirali njegove podmenije i njihove prečice. Možete dvaput kliknuti na bilo koju kontrolu da biste kreirali njen podrazumevani obrađivač događaja, gde se konvencija imenovanja **ControlName_EventName** koristi za prepoznavanje obrađivača događaja.

Takođe možete dodati obrađivače događaja iz gornjih padajućih lista uređivača koda – Form Code, tako što ćete izabrati naziv kontrole sa leve liste (recimo „**Button1**“) i kliknuti na naziv događaja sa desne liste (recimo „**OnClick**“), tako da će ovaj pod-me ni biti dodat u uređivaču koda:

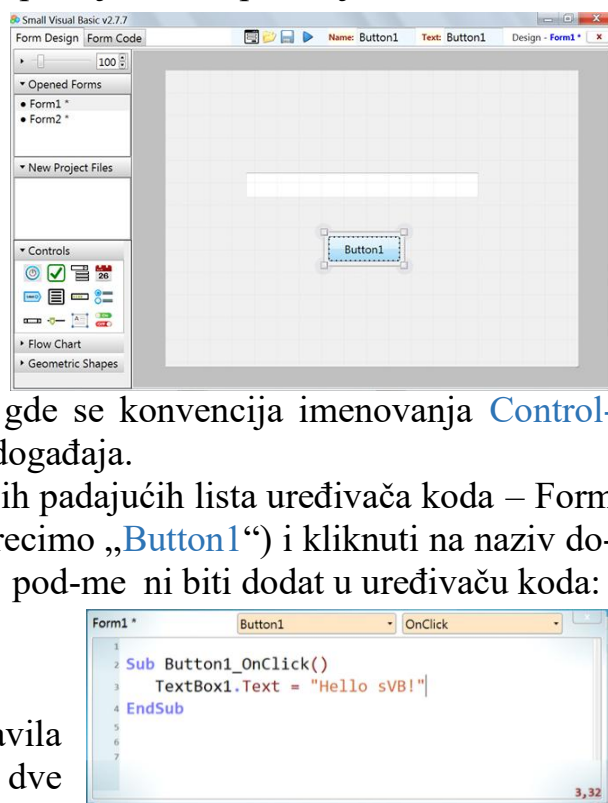
```
Sub Button1_OnClick()
```

```
EndSub
```

Kao i SB, sVB je dinamički jezik koji koristi ista pravila sintakse kao i za Visual Basic sintaksu kada dodate ove dve izjave na vrh VB kodne datoteke (ovaj kod se ne koristi u sVB):

```
Option Strict Off  
Option Explicit Off
```

sVB takođe ima neke druge VB sintaksne funkcije koje SB ne podržava, kao što su:



1. sVB ima neke OOP funkcije kao što je korišćenje imena kontrola kao objekata za direktan pristup njihovim svojstvima i metodama pomoću tačke, kao što je:

```
TextBox1.Text = "Zdravo sVB!"
```

TextBox1 je i dalje dinamička promenljiva koja sadrži string ključ kontrole (koji je „form1.textbox1“) i možete ga pročitati ili promeniti u bilo koju vrednost u bilo kom trenutku (što se naravno ne preporučuje), ali automatsko dovršavanje u editoru i IntelliSense vam daju osećaj da je objekat, a sVB kompajler prevodi ovu sintaksu sličnu objektu u normalnu SB sintaksu kao što je:

```
TextBox.SetText(TextBox1, „Zdravo sVB!“)
```

2. sVB vam omogućava da koristite **Me** za referencu na trenutni obrazac.

3. sVB vam omogućava da koristite simbol kontinuiteta linije **_** da biste podelili dugačku komandnu liniju na bilo kojoj poziciji koju želite (osim oko tačke i !). Na primer:

```
TextBox1.Text = „Zdravo sVB! “ + _  
„Ovo je moj prvi program.“
```

4. U SB, sve promenljive su globalne, što znači da cela datoteka ima samo jedan opseg promenljive. sVB se ponaša kao VB, gde su promenljive definisane u potprogramu lokalne za njega, dok su promenljive definisane na nivou datoteke (van potprograma i funkcija) globalne za celu datoteku. Pogledajte *Kako rešiti probleme sa SB promenljivim u sVB*.

5. sVB vam omogućava da deklarišete funkcije koje vraćaju vrednosti.

6. sVB omogućava potprogramama i funkcijama da deklarišu parametre.

7. sVB ima **ForEach** naredbu za petlju kroz nizove, što je slično VB For Each.

8. sVB koristi **ExitLoop** za izlaz iz For i While petlji, što je slično Exit For i Exit While u VB.

Ali imajte na umu da postoji nekoliko sintakskih razlika između SB/sVB i VB, kao što su:

1. sVB koristi **array[i]** za indeksiranje niza umesto array(i) u VB.

2. sVB niz je zapravo rečnik, tako da možete koristiti string ključeve umesto numeričkih indeksa, i zato indeksi nizova ne moraju biti kontinuirani niti uređeni, pa čak mogu biti i negativni kao što je **array[-1]**, jer kompajler zapravo tretira -1 kao string ključ "-1", tako da zapravo radite sa array["-1"].

3. Dve ključne reči u VB-u (kao što su For Each, End If, End While, End Sub i End Function) su kombinovane u jednu reč u sVB-u: (**ForEach**, **EndIf**, **EndWhile**, **EndSub** i **EndFunction**).

sVB i Visual Basic .NET

sVB se zapravo može nazvati sVB.NET, jer podržava neke VB.NET sintaksne karakteristike kao što su:

1. sVB dozvoljava implicitni kontinuitet linije pre ili posle nekih simbola kao što su zarez, zagrade i aritmetički operatori:

```
TextBox1.Text = "Zdravo sVB! "  
+ "Ovo je moj prvi program."
```

2. sVB koristi inicijalizatore niza **{}** da bi dodao elemente u niz:

```
a = {1, 2, 3, 4}
```

3. sVB podržava literale datuma i vremena kao što su:

```
d = #1/31/2023 13:00:00#
```

Imajte na umu da gornji literal koristi format datuma engleske kulture.

Literali datuma su fleksibilniji u sVB nego u VB.NET-u, jer sVB podržava bilo koji važeći

format datuma engleske kulture koji se može analizirati pomoću [.NET Framework DateTime.Parse](#) metode, kao što je [#31 Jan 2023#](#).

sVB čak ima i vremenski raspon kao što je [#+3:15:00#](#) koji VB.NET ne podržava!

4. sVB koristi neka poznata .NET unapred definisana nabiranjaja kao što su [Colors](#) i [Keys](#) sa podrškom za automatsko dopunjavanje:

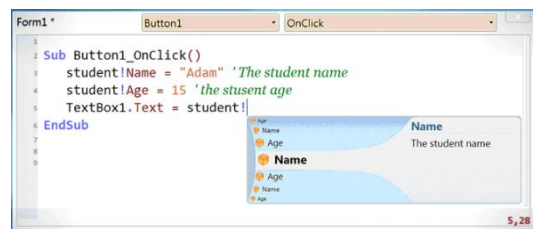
```
me.BackColor = Colors.AliceBlue
```

5. sVB koristi [ContinueLoop](#) u [For](#) i [While](#) petljama, što je slično Continue For i Continue While u VB.NET-u.

6. sVB koristi operator zaključavanja rečnika [!](#) za pristup ključevima niza, što je slično rečnicima u VB.NET-u:

```
student["ID"] = 1  
TextBox1.Text = student!ID
```

Takođe, ovaj operator se može koristiti za dodavanje dinamičkih svojstava nizovima, kao što je [Expando](#) objekat u VB.NET-u, ali sVB pruža dinamička svojstva za automatsko dopunjavanje, a možete čak koristiti i komentare da biste pružili [IntelliSense](#) informacije o njima (slika desno):



7. sVB koristi zaključivanje tipa slično VB.NET-u, da bi detektovao tip promenljive na osnovu njene početne vrednosti. Ovo ne menja činjenicu da su sVB promenljive dinamički tipovi i mogu da sadrže bilo koju vrednost bilo kog tipa u bilo kom trenutku, ali zaključivanje početnog tipa promenljive omogućava kompajleru da je tretira kao objekat, tako da možete direktno pristupiti njenim svojstvima i metodama kao što to radite u VB.NET-u. Na primer:

```
n = 2  
x = " Test "  
TextBox1.AppendLine(n.Power(3))  
TextBox1.AppendLine(x.Trim())
```

Gornji kôd je skraćeni oblik ovog koda:

```
n = 2  
x = " Test "  
TextBox1.AppendLine(Math.Power(n, 3))  
TextBox1.AppendLine(Text.Trim(x))
```

Gornji primer možda nije toliko važan za vas, ali može biti kada sVB zaključuje tip promenljive iz izraza ili povratne vrednosti metode, posebno kada metoda vraća obrazac ili kontrolu, tako da sVB može tretirati promenljivu kao kontrolu, što olakšava direktno rukovanje njenim metodama i svojstvima:

```
list = Me.AddListBox("lstTest", 10, 10, 200, 300)  
list.AddItem({1, 2, 3, 4})
```

Ovo nekako stavlja sVB između dinamički tipiziranih i statički tipiziranih jezika!

8. sVB može paralelno izvršavati potprograme u više niti.

9. Izvorni kôd samog sVB-a je napisan u VB.NET-u, a sVB projekti se kompajliraju u IL kod, slično VB.NET i C# projektima! To znači da možete koristiti bilo koji reflektor da dekompileirate bilo koju .exe datoteku koju je kreirao sVB nazad u C# projekat!

10. sVB [WinForms](#) kontrole i njihove metode su veoma bliske onima iz VB.NET WinForms-ama.

11. sVB ima mali [UnitTest](#) frejmwork koji vam omogućava da pišete i pokrećete test funkcije za testiranje vašeg projekta.

12. sVB može da kreira sopstvene biblioteke klasa, pored mogućnosti korišćenja VB.NET-a i C#-a za kreiranje biblioteka za sVB.

sVB IDE i VS.NET IDE

sVB editor je izgrađen na bazi SB editora, sa mnogim poboljšanjima koja ga čine sličnijim malom VS.NET editoru, kao što su:

1. Kôd blokira automatsko dovršavanje, kao što je dodavanje **Then** i **EndIf** nakon pisanja **If**.
2. Isticanje identifikatora i navigacija pomoću **Ctrl+Shift+Up** ili **Ctrl+Shift+Down**.
3. Intellisense informacije o fokusiranim rečima sa linkom za prelazak na definiciju.
4. Potpuno funkcionalan debager koji vam omogućava da pratite izvršavanje vašeg koda tokom rada.

sVB i WPF

sVB je izgrađen korišćenjem Window Presentation Foundation (WPF) okvira, kako bi imao napredne IDE funkcije. Na primer, dizajner obrazaca – Form Design vam omogućava da lako kreirate napredne vizuelne efekte, poput rotiranja i iskrivljavanja kontrola, a dijalog za boje – color dialog vam omogućava da koristite čvrste četkice, linearne i radijalne gradijentne četkice, kao i četkice za pločice i slike za crtanje ivice, pozadine i prednjeg plana kontrole, što vam omogućava da promenite oblike oblika i kontrola.

Na primer, kada pokrenete projekat *FormShape* iz fascikle sa primerima, videćete ovaj eliptični oblik, gde njegove providne ivice zapravo ne postoje, tako da možete kliknuti na bilo koji drugi prozor kroz njih! (slika desno gore)

Takođe, kada pokrenete projekat *ListBox2* iz foldera sa primerima, videćete ovu eliptičnu dugmad (slika desno):

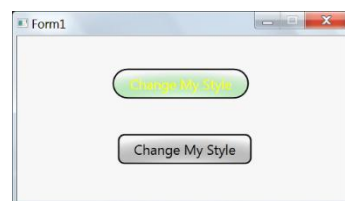
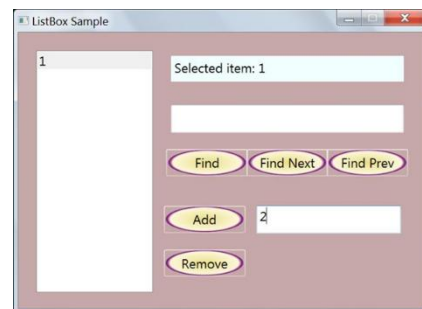
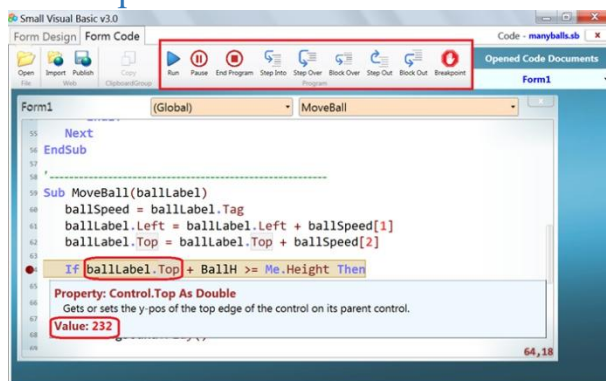
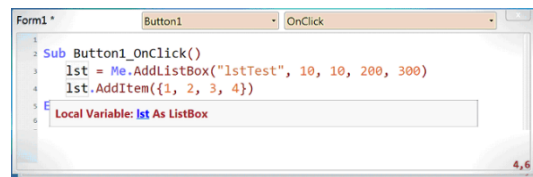
Osim toga, možete koristiti metode **Control.SetResourceDictionary** i **Control.SetStyle** da biste učitali rečnik resursa iz eksterne XAML datoteke, da biste primenili napredne stilove na formu i bilo koje druge kontrole koje želite da ciljate! (slika ddesno)

Rešavanje problema sa SB promenljivim u sVB-u

Najčešći razlog za kršenje Small Basic kodova u Small Visual Basic-u su domeni promenljivih. U SB-u, sve promenljive su globalne za celu datoteku, što je loša praksa za decu i učiniće da pate kada pređu na prave programske jezike. sVB je ovo rešio, tako da su promenljive definisane unutar potprograma privatne za ovaj potprogram i nevidljive su za druge potprograme.

Dakle, kada prenosite bilo koji SB kôd u sVB, promenljive na koje se sVB žali treba inicijalizovati na početku datoteke, postavljanjem na 0 za brojeve, "" za stringove i {} za nizove.

Takođe se preporučuje konvertovanje promenljivih koje se koriste za slanje podataka



potprogramama u parametre i konvertovanje promenljivih koje primaju podatke iz potprograma u povratne vrednosti funkcija. Na primer, ovaj SB kod:

```
X = 10
Y = 20
Sum()
TextWindow.WriteLine(z)
Sub Sum
z = X + Y
EndSub
```

treba prepisati ovako u sVB:

```
Z = Sum(10, 20)
TextWindow.WriteLine(Z)
Function Sum(x, y)
Return x + y
EndFunction
```

Ili čak možete i da se rešite z:

```
TextWindow.WriteLine(Sum(10, 20))
```

Ovo je ispravan način organizovanja promenljivih u vašem programu, što čini kôd čistijim, kraćim i efikasnijim.

sVB za decu

Pravedno je pitati se kako predstaviti sVB deci, iako deluje veoma veliko u poređenju sa SB.

U stvari, sVB može biti veoma jednostavan za decu. Sve što vam je potrebno je da pripremite nekoliko atraktivnih slika, prevučete ih i ispustite na dizajner obrazaca – Form Designer da biste prikazali oznake i koristite metode animacije da biste napisali neke kratke i zabavne aplikacije koje mogu da zainteresuju malu decu (pogledajte animaciju i Džeri projekte u fascikli sa primerima).

Možete početi sa aplikacijama bez koda, koristeći dizajner obrazaca za prikazivanje nekih oblika i slika, koristeći dijalog za boje da promenite njihove pozadinske boje.

U sledećem koraku možete dodati nekoliko dugmadi aplikaciji, svako sa jednom linijom koda koja animira lokaciju oblika, boju ili transparentnost.

U sledećem koraku možete uvesti promenljive, ali izbegavajte IF naredbe i petlje dok se dete ne navikne na jezik.

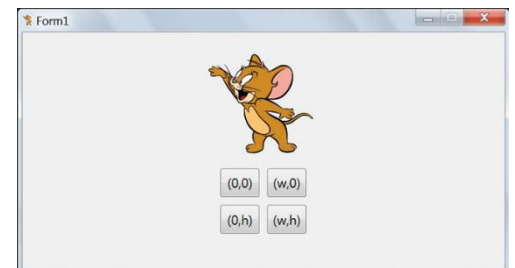
U stvari, počeo sam da pišem seriju knjiga „Small Visual Basic programer za decu“ da bih primenio te smernice. Prve 4 knjige ove serije su već objavljene na Amazonu.

Takođe sam objavio kurs za podučavanje sVB-a za decu na Jutjubu, ali je na arapskom. Sadrži više od 80 video snimaka (svaki od prosečno 10 minuta) i mislim da je programerima i nastavnicima lako da ih gledaju čak i ako ne razumeju arapski, jer će vizuelni dizajn i mali sVB kôd koji se koristi, uz pomoć ove knjige, biti lako razumljivi, tako da mogu da shvate ideju i da grade na njoj.

Podstičem Jutjubere da uvedu takav kurs na engleskom i drugim jezicima.

Zaključci

1. Small Visual Basic nije baš MALI, jer ga možete koristiti za kreiranje velikih projekata koji sadrže desetine obrazaca. Deo „Small“ u nazivu je samo da bi se odrazilo da je izgrađen na Small



Basic-u.

2. sVB kombinuje najbolje od SB, VB, VB.NET, VS.NET IDE, WinForms i WPF okvira.
3. sVB je jednostavna, lagana verzija .NET platforme, sa instalaterom od samo 20 MB za preuzimanje i minimalnim hardverskim zahtevima za pokretanje!
4. Svrha sVB-a je da bude atraktivan za decu i početnike da uče programiranje, a zatim lako pređu na .NET platformu.
5. Učenjem sVB-a, vi ste samo jedan korak napred ka učenju VB.NET-a.
6. sVB vas obučava kako da koristite Windows Forms frejmwork (WinForms), a takođe otvara mali prozor za pregled nekih neverovatnih WPF mogućnosti, tako da biste kasnije mogli biti zainteresovani da naučite jednu od XAML porodičnih tehnologija kao što su WPF, MAUI ili WinUI.